

Practical 5 – Problem sheet

This exercise sheet discusses two topics: composite data types (exercises 1–2) and character strings (exercises 3–5). You can proceed in any order, but be sure to work on both topics – either in the session or at home.

The practical should also be used as an exercise to follow the code style conventions introduced in the lectures, which will be relevant for the next project:

- Place all code block delimiters – { and } – on separate lines.
- Within code blocks, indent the code by 4 spaces with respect to the surrounding code.
- Choose all names for variables, parameters, and functions to start with a lowercase character.

Please feel free to ask for feedback on your code regarding these items. The BlueJ menu item *Edit* → *Auto-layout* may be helpful.

Exercise 1 S

In this exercise, you will practice some basic operations related to composite data types, as discussed in Lesson 17; see in particular the classes `BankAccount` and `UsingBankAccounts` there. Here, specifically, we will create a data type that contains the data for an employee.

- Create a composite data type `Employee`. To that end, select “Edit → New class” from the menu; enter the class name, and click “OK”. The class will be filled with some sample code, which it is best to delete. Then create three fields in the class: `firstName` and `lastName`, both of type `String`, and `salary` of type `int`. (The latter is supposed to contain the employee’s salary in £ per month.) Run the unit test `EmployeeTestA`. *Note:* This part of the exercise does not create any functional code yet. It just declares the data structure.
- In the class `EmployeeActions` (which is given), create a function `sampleEmployee()`, with no parameters, that returns an `Employee` record corresponding to the employee Joe Bloggs, with a salary of £1600.
- Again in the class `EmployeeActions`, create a procedure* `increaseSalary()` which takes an employee record and an integer (say, p) as its input, and modifies the employee record so that the salary is increased by p pounds.

Exercise 2 I

In Lesson 17, we introduced a composite data type `ComplexNumber` which was used to store a complex number in floating point format. The class `ComplexNumber` is included in the code template. The lecture also introduced a function `abs` that computes the absolute value of a complex number, and a function `add` that computes the sum of two complex numbers. These functions are also included in the code template (class `ComplexComputation`).

Your task is to extend this a little. Inside `ComplexComputation`, write:

- a function `subtract` that takes two complex numbers z and w as arguments, and returns $z - w$;

*that is, a function without return value, which is coded by declaring the return type as `void`

- (b) a function `conjugate` that takes one complex number z as its argument, and returns the complex conjugate $\bar{z}$;
- (c) a function `multiply` that takes two complex numbers z and w as arguments, and returns $z \cdot w$;
- (d) a function `divide` that takes two complex numbers z and w as arguments, and returns z/w .

Exercise 3 (S)

This exercise uses some standard features of the `String` class, as discussed in Lesson 18. You may want to review the lecture slides first, and/or read the documentation of the class. In particular, the methods `charAt(...)`, `indexOf(...)` and `substring(...)` are helpful in the following.

Within the class `StringExamples`, write the following functions, which all take a `String` value as their only parameter. You can assume in all cases that the input parameter is not `null`.

- (a) A function `containsE(...)`, that checks whether the string contains the lowercase letter 'e'. The return value should be of type `boolean`, namely, it should be `true` if the string contains an 'e' and `false` otherwise.
- (b) A function `countE(...)`, which counts the number of occurrences of the letter 'e' in the string, and returns this number as `int`.
- (c) A function `splitAtSpace(...)`, which looks for the first occurrence of a space character (' ') in the string, replaces this character with three hyphens (---), and returns this modified value. If the input string does not contain a space character, it should be returned unmodified.

Exercise 4 (I)

This exercise expands further on the use of the class `String`. Please use the class `Palindromes` provided.

- (a) Write a function `reverse` that takes a `String` as its input, reverses the order of characters, and returns the result as a `String`. Example: if the input string is "Test", the output should be "tseT". If the input is `null`, so should the output be.
- (b) Write a function `isPalindrome` that tests whether a string (supplied as the only parameter) is a palindrome, that is, whether it is equal when read from left to right and from right to left. Example: "kayak" is a palindrome. The return type of the function should be `boolean`. Your function should ignore uppercase/lowercase differences when testing; i.e., "Kayak" is a palindrome as well. Also, it should consider `null` as *not* a palindrome.

Exercise 5 (A)

Pig Latin is a language game in which English words are modified as follows:

- If the word starts with a vowel, we append "way" to it.

- If the word starts with one or more consonants, we move the consonants to the end and append “ay” to the result.
- Otherwise, the word goes unmodified,

Vowels are `aeiou`. The letter `y` counts neither as a consonant nor as a vowel.

Some examples: “owl” becomes “owlway”, “duck” becomes “uckday”, “grouse” becomes “ousegray”.

Write a function `translateToPigLatin` that takes an English text (as `String`) as a parameter, translates all words in it according to the rules above, and returns the result as a `String`. Words are separated by spaces. You can assume that only letters and spaces occur in the text.[†] Before translating, convert all characters to lowercase first.

Iway opehay you avehay unfay ithway isthay exerciseway!

[†]As an extension of the exercise, you might also want to handle digits, punctuation, etc.